COLUMBIA UNIVERSITY
School of Professional Studies

Module 1: Course Overview and Basics of Generative AI

Summer 2026

Outline

- Course Overview
- Intuition
- Basics of Probability
- Natural Language Sequence Modeling
- Word and Sentence Embeddings

Course Overview

Frameworks

There are many frameworks and libraries used in Generative AI and Deep Learning. For greatest flexibility we will use mostly well-established lower-level frameworks.

- **Docker -** Platform that allows building and running applications in isolated environments called **containers**. Containers make it easy to package applications with all of their dependencies, ensuring they run reliably across different systems (e.g. locally or on the AWS/Google Cloud).
- **FastAPI -** Modern, fast (high-performance), web framework for building APIs with Python.
- **PyTorch and TensorFlow -** open-source frameworks for numerical computations used in Deep Learning.

Deep Learning Architectures

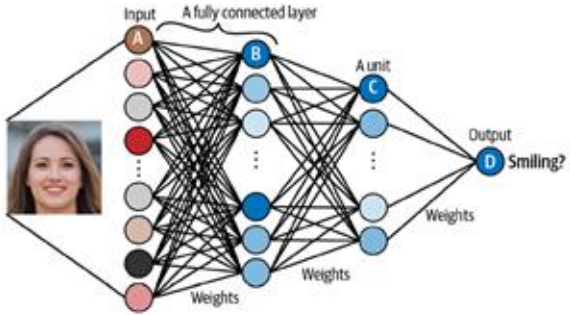


Image Source: Foster, D. (2024). Generative deep learning: Teaching machines to paint, write, compose, and play (2nd ed.). O'Reilly Media.

- **Convolutional Neural Networks (CNN) -** use special `convolutional layers` to detect spatial patterns and hierarchical features.
- **Variational Autoencoders (VAE) -** learn to encode/decode data into/from a latent space.
- **Generative Adversarial Networks (GAN) -** consist of a generator and a discriminator, where the generator creates realistic data and the discriminator learns to distinguish between real and generated samples.
- **Recurrent Neural Networks (RNN)-** neural networks designed for sequential data and containing feedback loops.
- **Transformers -** use self-attention mechanism to 'attend' to specific part of the input, allowing the model to detecting patterns in sequential data.

Advanced Models

- **Normalizing Flows** - generative models that invertibly map a simple probability distribution into a more complex one.
- **Energy Models** - use the idea of ‘energy’ borrowed from Physics, instead of probability distributions.
- **Diffusion Models -** generative frameworks that learn to denoise an image and can generate realistic looking images from noise.
- **Reinforcement learning -** instead of minimizing a cost function, in reinforcement learning an agent interacts with an environment to maximize rewards.

Intuition

What is Generative AI?

- In the simplest terms, **Generative AI models** create new data that is **similar** to a given dataset.
- Consider some names sampled from a dataset:

- Can you generate other similar names?

What is Generative AI?

There is often **no single correct answer** when it comes to recognizing patterns in data. Often, identifying the pattern becomes easier when you understand the **context** or the **hidden structure** behind the data.

For example:

- **What if the names come from an Astronomy textbook?** In this case, the names might correspond to celestial bodies like stars, constellations, or planets.
- **What if the names come from a Greek and Roman Mythology textbook?** Here, the names could be those of gods, goddesses, or mythological creatures.

The Context

Example with ChatGPT

- If we want to generate new data, it will depend on this context.
- The ability to create new but contextually consistent data is what makes Generative AI powerful for tasks like text completion, image synthesis, and even music

Complete the list of the following planets: Jupiter, Saturn, Pluto, Mercury, Mars. Give the answer in one sentence.

The complete list of planets, including the missing ones, is: **Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune, and Pluto (dwarf planet).**

Context Matters

Example with ChatGPT

Complete the list of the following planets: Jupiter, Saturn, Pluto, Mercury, Mars. Give the answer in one sentence.

The complete list of planets, including the missing ones, is: **Mercury, Venus, Earth, Mars, Jupiter, Saturn, Uranus, Neptune, and Pluto (dwarf planet).**

Complete the list of the following mythological figures: Jupiter, Saturn, Pluto, Mercury, Mars. Give the answer in one sentence.

The complete list of Roman mythological figures corresponding to the major gods is: **Jupiter, Saturn, Pluto, Mercury, Mars, Venus, Neptune, and Apollo.**

Add just enough uncertainty…

- Design of these powerful models requires **some understanding of probability theory.**
- The models can learn the **underlying probability distributions** of the data, allowing them to generate new samples.
- For example, in text generation the model estimates the **probability** of each word occurring given the previous ones. We can then sample from these probabilities to construct new words (or tokens) that follow the **same patterns** as the words in the training data.
- By leveraging probability theory, Generative AI models can **balance creativity and realism**, producing outputs that are both novel and contextually appropriate.

Basics of Probability

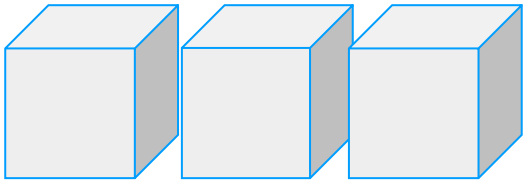
Basics of Probability

To discuss/predict future events, we need the concept of **Probability**. How should we define it?

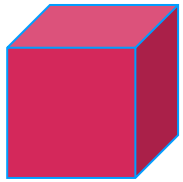
- Regular variables have one value at a time, e.g. **X = 5**.
- **Random** or **Stochastic** variables allow us to deal with unknown quantities/events and instead assign to each possibility a **probability** P.
- To be useful, this probability should follow certain rules.
- Let’s explore them with a game…

Treasure Game

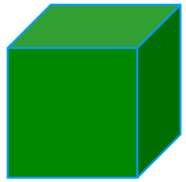
We will use **X** to describe our belief of where the treasure is.



- Let’s use a random variable **X** to model which one of the boxes contains a treasure.
- Don’t worry, I will give hints…
- Red box - no treasure

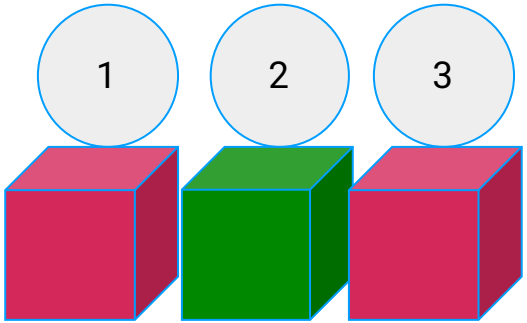


- Green box - treasure



Example 1

Here we know for certain that the treasure is in box 2.



Some rules:

- Use P(**X** = …) = 1 to mean that **X** = … is certain
- Use P(**X** = …) = 0 to mean that **X** = … is impossible.

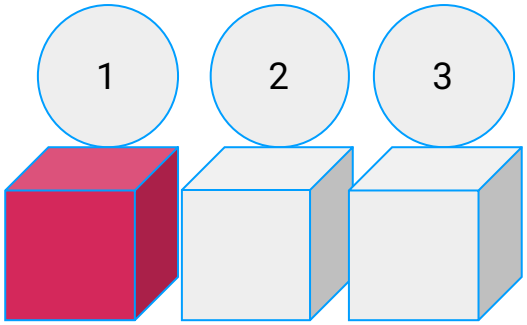
Let's define a random variable that encodes our belief about the treasure location:

- P (**X** = 1) = 0
- P (**X** = 2) = 1
- P (**X** = 3) = 0

Note: when we define a random variable we give probabilities of its values.

Example 2

Here, things are more interesting, the treasure could be in box 2 or 3.

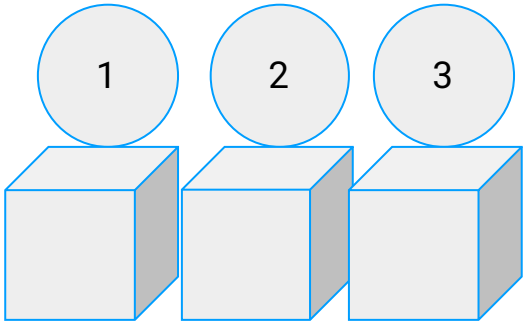


- Use P(**X** = …) between 0 and 1 to represent our belief of where the treasure is.
- P(X = **A** or **B**) = P(X = **A**) + P(X = **B**)

- What do we know for certain about X?
  - P (**X** = 1) = 0
  - P (**X** = 2 or 3) = 1
- Possible definition of X:
  - P (**X** = 1) = 0, P (**X** = 2) = ½, P (**X** = 3) = ½
- As long as we keep P(X = **2** or **3**) = P(**2**) + P(**3**) = 1, we can adjust the probabilities if we want to represent the belief that box 3 might be a bit more likely to contain the treasure, for example.

Example 3

Define a new random variable **Y** to describe our belief about the color of box 2.



Some more rules:

- Use P(**X** = … | **Y** = …) to mean:
  - Probability that **X** = … GIVEN THAT **Y** = …
- P (**X**=..., **Y** = …) probability that **X** = … AND **Y** = …
- P(**A** AND **B**) = P(**A** | **B**) P(**B**)
- P(**A** AND **B**) = P(**A**) P(**B**) if **A** and **B** are independent
- What do we know for certain about **X** and **Y**?
  - P (**X** = 1| **Y** = GREEN) = ?
  - P (**X** = 2 | **Y** = RED) = ?
  - P (**X** = 2 | **Y** = GREEN) = ?
  - P (**X** = 1 or **X** = 3 | **Y** = GREEN) = ?

Fundamental Rules of Probability

sum rule $$p(X) = \sum_{Y} p(X, Y)$$

product rule $$p(X, Y) = p(Y|X)p(X).$$

Bayes Theorem helps us to represent our belief about the world based on evidence and data:

$$p(Y|X) = \frac{p(X|Y)p(Y)}{p(X)},$$

Probability Practical

[Interactive Notebook]

Bayes Theorem is fundamental in calculating probabilities of latent variables based on observable data, for example localizing a self-driving car based on noisy sensor information. While we will not use it directly, the following activity will help us practice with concepts of probability and likelihood that will play an important role in the following modules.

Natural Language Sequence Modeling

Probabilities in Language Modeling

P(word1, word2, word3, …)

Some notation:

- Use P(“book”) as a shorthand for P($\mathbf{X}_i$ = “book”), where $\mathbf{X}_i$ is a random variable representing the ith word.
- So P(“New”, “York”) is the probability that P($\mathbf{X}_1$ = “New” and $\mathbf{X}_2$ = “York”), i.e. that the first word is “New” and the second word is “York”.

Let’s come up with rough estimates (rank most probable -> least probable)

- P(“Sam”, “went”, “to”, “the”, “supermarket”)
- P(“Sam”, “car”, “supermarket”)
- P (“Sam”, “swam”, “to”, “the”, “supermarket”)

Probabilities in Language Modeling

P(word | context)

Some notation:

- P(“tea” | “green”) is the probability that P($\mathbf{X}_2$ = “tea” | $\mathbf{X}_1$ = “green”), i.e. that the second word is “tea” given that the first word is “green”.
- P (“Paris” | “The”, “capital”, “of”, “France”, “is”) is a shortcut for P($\mathbf{X}_6$ = “Paris” | $\mathbf{X}_1$ = “The”, $\mathbf{X}_2$ = “capital”, $\mathbf{X}_3$ = “of”, $\mathbf{X}_4$ = “France”, $\mathbf{X}_5$ = “is”)

Let’s give rough estimates (high probability or low probability)

- P(“Toyota” | “My name is”) = LOW OR HIGH?
- P(“John” | “My name is”) = LOW OR HIGH?

Probabilities in Language Modeling

Markov Assumption

Example:
[Interactive Notebook]

Remember the product rule?

- $P(\mathbf{X}_1 \ldots \mathbf{X}_n) = P(\mathbf{X}_1) * P(\mathbf{X}_2 \mid \mathbf{X}_1) * P(\mathbf{X}_3 \mid \mathbf{X}_{1:2})$ ….
- This means to model probabilities correctly we have to track all of the context from the very first word.

To simplify things, it is common to add the **Markov Assumption:**

- $P(\mathbf{w}_n \mid \mathbf{w}_{1:n-1}) = P(\mathbf{w}_n \mid \mathbf{w}_{n-1})$
- I.e. each consecutive word only depends on the previous one - this is the **bigram model**.
- Can also use several previous ones: **n-gram model.**
- Modern LLM systems can have context sizes of 128,000 tokens or more.

Word and Sentence Embeddings

Word Embeddings

- In **Methods and Frameworks II**, you explored how to generate numeric representations of words using a **Document-Term Matrix**—also known as **Vector Representations**—which map words into vectors within an n-dimensional space based on the term frequency of their occurrence in a given document..

Document-Term Matrix:

	and	bark	become	bright	...	tree	warm	water
0	0	0	0	0	...	0	0	0
1	0	0	0	0	...	0	0	0
2	0	0	0	0	...	1	0	0
3	0	1	0	0	...	0	0	0
4	1	0	1	0	...	0	0	0
5	0	0	0	0	...	0	0	0
6	0	0	0	0	...	0	0	0
7	0	0	0	0	...	1	0	0
8	0	0	0	1	...	0	1	0
9	0	0	0	0	...	0	0	0

Sparse Embeddings

However, these representations come with several limitations:

- **High Dimensionality**: The dimension of each vector corresponds to the size of the vocabulary obtained after preprocessing steps like stemming and stopword removal. This often results in extremely sparse and high-dimensional vectors, making computations resource-intensive.
- **Lack of Contextual Information**: These vectors treat words in isolation, so the vector representation of *"table"* is as different from *"desk"* as it is from *"orange"*—even though *"table"* and *"desk"* are semantically related. This inability to capture context limits their effectiveness in understanding word meanings.

Word2Vec

- One of the pivotal efforts to address these challenges was a highly influential 2013 paper from Google titled, **“Efficient Estimation of Word Representations in Vector Space.”**
- The core idea was to project high-dimensional word representations into a lower-dimensional space, ensuring that words with similar meanings are mapped to vectors that are close according to a similarity measure, such as Euclidean distance or cosine similarity.
- These mappings are called **“embeddings,”** a term borrowed from the field of **Topology** in mathematics. Conceptually, these embeddings can also be understood as **latent representations** of the original vectors, capturing the underlying semantic relationships between words.

Sentence Embeddings and Information Retrieval

- For tasks such as **Information Retrieval** and **Retrieval-Augmented Generation (RAG)**—topics we will explore in the next module—we need to construct **embeddings** not just for individual words, but for entire sentences and even larger documents.
- One straightforward approach to achieve this is to **average the embedding vectors** of all the words in the document. By aggregating the word embeddings, we can create a single, unified vector that captures the overall semantic meaning of the entire sentence or document.

Embeddings

Example:
Interactive Notebook

- In modern NLP systems text is divided into subunits called **tokens**, rather than words. For now we will work with word tokens to keep things simple.
- **Word2Vec** paved the way to many other embedding implementations.
- For simplicity we will start with **spacy** implementation and later experiment and learn how to implement more advanced embeddings provided by the latest Large Language Models (LLM).